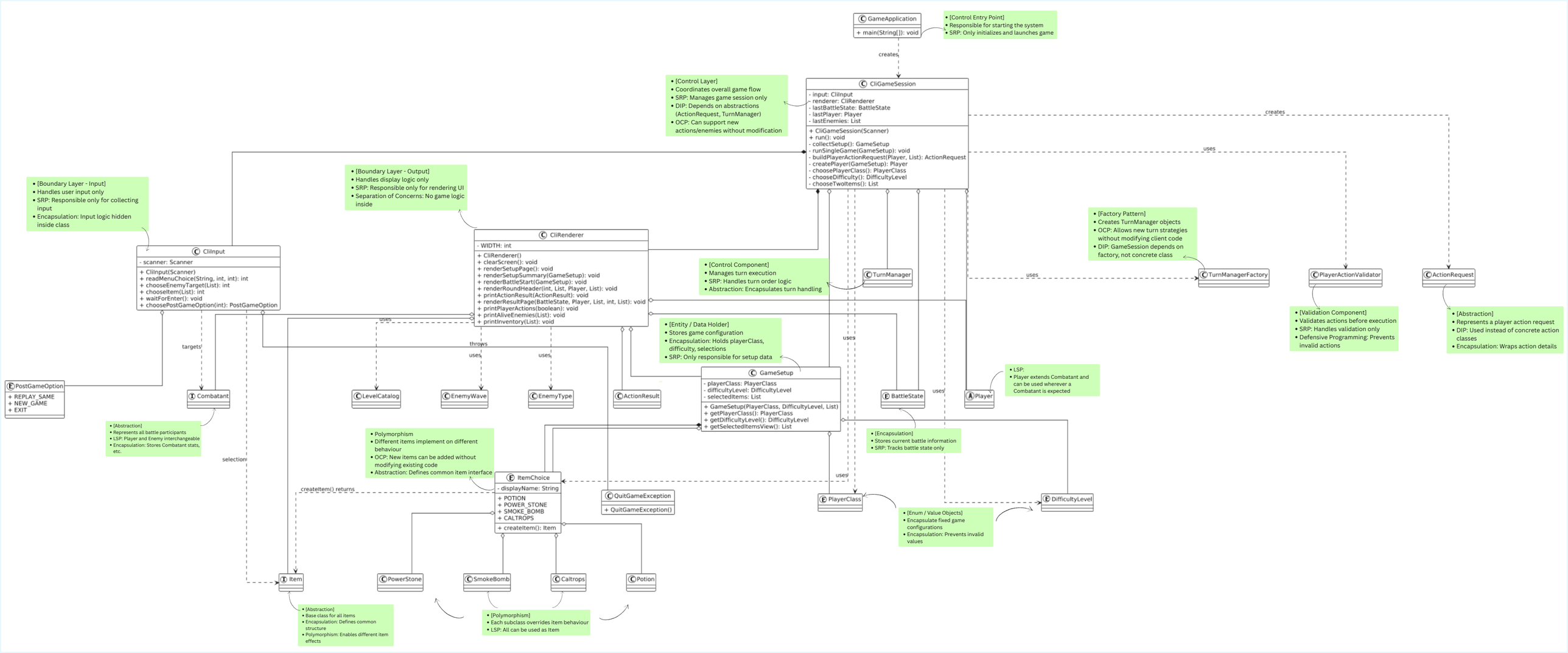
GameApplication
+ main(String[]): void
• [Control Entry Point]
• Responsible for starting the system
• SRP: Only initializes and launches game
creates
CliGameSession
- input: CliInput
- renderer: CliRenderer
- lastBattleState: BattleState
- lastPlayer: Player
- lastEnemies: List
+ CliGameSession(Scanner)
+ run(): void
- collectSetup(): GameSetup
- runSingleGame(GameSetup): void
- buildPlayerActionRequest(Player, List): ActionRequest
- createPlayer(GameSetup): Player
- choosePlayerClass(): PlayerClass
- chooseDifficulty(): DifficultyLevel
- chooseTwoItems(): List
• [Control Layer]
• Coordinates overall game flow
• SRP: Manages game session only
• DIP: Depends on abstractions (ActionRequest, TurnManager)
• OCP: Can support new actions/enemies without modification
creates
uses
• [Boundary Layer - Input]
• Handles user input only
• SRP: Responsible only for collecting input
• Encapsulation: Input logic hidden inside class
CliInput
- scanner: Scanner
+ CliInput(Scanner)
+ readMenuChoice(String, int, int): int
+ chooseEnemyTarget(List): int
+ chooseItem(List): int
+ waitForEnter(): void
+ choosePostGameOption(int): PostGameOption
• [Boundary Layer - Output]
• Handles display logic only
• SRP: Responsible only for rendering UI
• Separation of Concerns: No game logic inside
CliRenderer
- WIDTH: int
+ CliRenderer()
+ clearScreen(): void
+ renderSetupPage(): void
+ renderSetupSummary(GameSetup): void
+ renderBattleStart(GameSetup): void
+ renderRoundHeader(int, List, Player, List): void
+ printActionResult(ActionResult): void
+ renderResultPage(BattleState, Player, List, int, List): void
+ printPlayerActions(boolean): void
+ printAliveEnemies(List): void
+ printInventory(List): void
• [Control Component]
• Manages turn execution
• SRP: Handles turn order logic
• Abstraction: Encapsulates turn handling
TurnManager
• [Factory Pattern]
• Creates TurnManager objects
• OCP: Allows new turn strategies without modifying client code
• DIP: GameSession depends on factory, not concrete class
uses
TurnManagerFactory
PlayerActionValidator
ActionRequest
• [Validation Component]
• Validates actions before execution
• SRP: Handles validation only
• Defensive Programming: Prevents invalid actions
• [Abstraction]
• Represents a player action request
• DIP: Used instead of concrete action classes
• Encapsulation: Wraps action details
targets
uses
throws
uses
uses
uses
• [Entity / Data Holder]
• Stores game configuration
• Encapsulation: Holds playerClass, difficulty, selections
• SRP: Only responsible for setup data
GameSetup
- playerClass: PlayerClass
- difficultyLevel: DifficultyLevel
- selectedItems: List
+ GameSetup(PlayerClass, DifficultyLevel, List)
+ getPlayerClass(): PlayerClass
+ getDifficultyLevel(): DifficultyLevel
+ getSelectedItemsView(): List
PostGameOption
+ REPLAY_SAME
+ NEW_GAME
+ EXIT
Combatant
LevelCatalog
EnemyWave
EnemyType
ActionResult
BattleState
uses
Player
• LSP:
• Player extends Combatant and can be used wherever a Combatant is expected
• [Abstraction]
• Represents all battle participants
• LSP: Player and Enemy interchangeable
• Encapsulation: Stores Combatant stats, etc.
selection
• Polymorphism
• Different items implement on different behaviour
• OCP: New items can be added without modifying existing code
• Abstraction: Defines common item interface
• [Encapsulation]
• Stores current battle information
• SRP: Tracks battle state only
uses
ItemChoice
- displayName: String
+ POTION
+ POWER_STONE
+ SMOKE_BOMB
+ CALTROPS
+ createItem(): Item
createItem() returns
QuitGameException
+ QuitGameException()
PlayerClass
DifficultyLevel
• [Enum / Value Objects]
• Encapsulate fixed game configurations
• Encapsulation: Prevents invalid values
Item
PowerStone
SmokeBomb
Caltrops
Potion
• [Abstraction]
• Base class for all items
• Encapsulation: Defines common structure
• Polymorphism: Enables different item effects
• [Polymorphism]
• Each subclass overrides item behaviour
• LSP: All can be used as Item